资产负债管理 · 反向测算决策中心

已载入**一套默认方案 + 一份示例数据**，可直接点「开始测算」；左侧参数分三类： 目标参数 目标指标、 算法调参参数 权重/红线/引擎参数、 五大引擎前置参数 科目系数（接口传入，非算法结果）。以后替换为真实决策中心接口即可。

参数配置 | 处理结果 | 五大引擎公式 | 流程图 | 接口文档

📖 接口文档（供其他系统调用）

本服务即「算法 + 五大引擎」的推理接口。决策中心（前端）与外部系统都通过 HTTP 调用；以后替换为真实决策中心接口时，保持本契约即可。**每个请求字段已标注 PRCP 数据字典来源（表.字段）**：标【确定】= 字段名完全对应；标【建议】= 语义对应，落点待业务确认。

★ 线上网关统一调用（只支持 POST，body 用 action 区分动作）

线上网关只暴露一个 POST 入口，URL 固定打 **/v1/cabs-alm-optimization/predict**（网关剥前缀打到根 /）；**动作靠 body 里的 action 区分，不带 action 默认=提交**。本文档下方 GET 进度/结果仅本地直连可用，线上一律用此统一 POST。
⚠ **网关 JSON 解析器不支持 options 内嵌套对象**，acor/de/slsqp 参数必须写扁平键（见下表下方红框）。

动作	body（Content-Type: application/json）	返回
①提交·快速深度学习 mode=dl	`{"options":{"mode":"dl"}}`	code=0 + **data.task_id**（抄下来，下面两步用）
①提交·双引擎同时 蚁群+差分+深度学习（algorithm，各2轮）	`{ "targets":{"roe":0.0961,"cet1":0.09,"lcr":1.2,"nsfr":1,"nim":0.0155,"eve":0.057}, "options":{"mode":"algorithm","acor_max_iter":2,"de_max_iter":2} }`	code=0 + data.task_id；targets 六指标按需要改，不传用默认
①提交·单算法 蚁群+差分，不跑dl（acor）	`{"options":{"mode":"acor","acor_max_iter":2,"de_max_iter":2}}`	code=0 + data.task_id
②查进度 （带历史记录）	`{"action":"progress","task_id":"<提交返回的task_id>"}`	data.done / data.success / data.percent / data.stage / data.history_count / data.history（最近 50 条进度历史）；done:true,success:true 再取结果
③取结果 可选 fields 拆分	`{"action":"result","task_id":"<提交返回的task_id>"}` 可选加 `"fields":"best|candidates"`（必须竖线：括号数组会被网关 500 拒，逗号会被解析吞掉） → 按需只返回指定顶层键，避免全量响应超网关 ~4MB 缓冲；不传 fields = 全量返回；返回恒带 reqid/baseline/delta	code=202 未完成 / code=500 失败看 message / code=0 成功；fields 只返回指定键 + reqid/baseline/delta（这三个恒带、不受白名单裁剪），不存在的键自动忽略（如 acor 模式取 best_dl 返回 null）

⚠ **网关的 JSON 解析器不支持 options 里再嵌套对象**（写成 `acor:{"max_iter":2}` / `de:{...}` 会被整块丢弃，参数静默回退默认值，表现为"改了参数但跑的还是默认轮数"）。线上必须用扁平键：**acor_m / acor_k / acor_xi / acor_rho / acor_q / acor_max_iter / acor_warm_start / acor_xi_decay / acor_patience / acor_parallel**、**de_np / de_f / de_cr / de_max_iter / de_tol**、**slsqp_max_iter / slsqp_ftol / slsqp_top_n**。嵌套写法只在本地直连（/api/abm/calculate、/v1/{id}/predict）有效。

成功后结果怎么取（外层 data）

要取的	路径 / 含义
综合总分	**data.best.score**：0~500 分制总分（各指标分段得分累加）
等级	**data.best.grade**：A≥400 / B≥300 / C≥200 / D（<200）
六指标实测值	**data.best.indicators**：cet1 / nim / lcr / nsfr / roe / eve 本次实测值（DL 路线取 data.best_dl.indicators）
目标达成对比	目标值 = **本次提交的 targets.***（未传用内置默认，GET /api/abm/config_schema 的 targets_default）；实际值 = **data.best.indicators.***；得分/命中段位 = **data.best.detail.details[]**（current_value/score/segment_desc/seg_order/higher_is_better）；达标判定 = 按 higher_is_better（1=越高越好）比较实际值 vs 目标值。⚠ **默认值分入口**：/api/abm/calculate 未传 = 0.0961 那组；网关入口未传 = targets_default（0.12 那组），见下方黄框

评分分项明细	**data.best.detail.details[]**：每个指标命中的段位与得分，字段含义见下表
月度指标（12 个月逐月）	**data.best.monthly**：{cet1/nim/lcr/nsfr/roe/nii/eve 各 12 个值 M1~M12}（DL 路线取 data.best_dl.monthly）
月度科目调整明细	**data.best.monthly_actions**：每月 12 账户的 ratio/t0_balance_yi/monthly_increment_yi/cum_increment_yi（DL 路线取 data.best_dl.monthly_actions）
12×65 增量动作矩阵	data.best.matrix_12x64（12行×65列，单位元；键名为历史命名）
期末余额表 / 现金流矩阵	data.best.final_bs_matrix / data.best.cashflow_64_matrix
完整月度资产负债表	data.best.balance_matrix_full（37 科目 × 25 月）+ account_names / account_codes / n_months / *_cols 列表头
全部候选（含来源）	data.candidates[]：source(acor|de|dl) / score / grade / indicators / plan_table / detail
迭代轨迹（收敛曲线）	data.trace[]：每代 {algo:'ACOR'|'DE', iter, best, top3_plans[轻量]}——画 ACOR/DE 收敛曲线取 algo/iter/best
⚠️ 完整全量明细	**data.result_file**（string）：本次任务**完整 payload** 落盘路径（容器/NAS 内，如 /log/result-90c7a91c.json）。响应为轻量版（见下文"响应瘦身说明"）：**只有 real_engine.cashflow 全量矩阵（现金流行矩阵）被裁到此文件**；monthly / monthly_actions / engine 中间量响应里仍有
📦 按需拆分取（fields 白名单）	同一 result 接口支持 **fields** 参数按需取顶层键（网关 POST body 加 `"fields":"best|candidates"`——必须竖线，括号数组会被 500 拒、逗号会被网关吞；本地 GET 加 `?fields=best|candidates` 或 `?fields=best,candidates`），每块都远小于网关 4MB 缓冲：`fields:"best"` ≈0.55MB / `"candidates"` ≈1.35MB / `"trace"` ≈1.85MB / `"best_dl"` ≈0.55MB（默认双引擎线上体积口径）；只取 `best` 时 = data.best + data.baseline + data.delta + data.reqid 四键（后三键恒带）。不传 fields = 全量（超限风险）

📱 处理结果页面 ↔ 接口请求与字段映射（决策中心前端实际调用）

前端「处理结果」各 tab 的全部数据都来自**同一个 result 接口**（本地 **GET /api/abm/result/{task_id}**；网关等价 POST /v1/cabs-alm-optimization/predict，body {"action":"result","task_id":"..."}）。下面把前端每个展示块 → 它实际读的字段路径列全，方便按图索骥。

页面动作 / 展示块	前端请求 & 接口字段路径
① 提交测算（运行按钮）	**POST /api/abm/calculate**，body = {targets:{cet1,nim,roe,eve,nsfr,lcr}, options:{mode:'acor'/'algorithm'/'dl', ...}} → 返回 code=0 + data.task_id（页面存为 taskId）
② 进度条 / 状态	**GET /api/abm/progress/{task_id}** → data.stage / percent / message / done / success / cancelled / history（最近 50 条）
③ 全部历史（折叠面板）	**GET /api/abm/progress/{task_id}/history** → data.history[] 全量，每条 {stage, percent, message, ts}（不截断）
④ 取结果渲染全部 tab	**GET /api/abm/result/{task_id}** → code=0 + **data**（result 对象，下述字段都在 data 下）；code=202 未完成 / 500 失败 / 404 不存在
⑤ 取消	**POST /api/abm/cancel/{task_id}**
算法 tab（数据源 = data.best）	
冠军分数	data.best.score（0~500 分）
评级 + 起点来源	data.best.grade；起点来源 = 从 data.candidates 过滤 source!=dl 后按 score 取最大者的 .source（acor=蚁群起点精修 / de=差分起点精修）
六指标实测表	data.best.indicators.{cet1, lcr, nsfr, nim, roe, eve, ibl, lr}；NII(亿) 取 data.best.engine.outputs.nii
目标达成对比	实际值 = data.best.indicators.*；目标值 = 本次提交的 targets.*；得分/段位 = data.best.detail.details[]（按 higher_is_better 判定达标）
摆布方案表	data.best.plan_table[]：13 项 {name, key, side, ratio, scale, mode}（x 向量=data.best.x 与此 ratio 一一对应）
月度指标（M1~M12）	data.best.monthly.{cet1, nim, lcr, nsfr, roe, nii, eve}（各 12 个值）
月度科目调整明细	data.best.monthly_actions[12]：每月 {month, items:[{account_code, account_name, ratio, t0_balance_yi, monthly_increment_yi, cum_increment_yi}]}
12×65 增量动作矩阵	data.best.matrix_12x64（12 行 × 65 列）+ data.best.matrix_12x64_cols（列名）+ data.best.plan_table（行科目）
完整月度资产负债表	data.best.balance_matrix_full（37 × n_months，算法路线 25 月）+ data.best.account_names / account_codes / data.best.n_months
张涛完整账户表（期末）	data.best.final_bs_matrix（37 × 67 = 科目代码+期末余额+65 桶剩余现金流）+ data.best.final_bs_matrix_cols（动态列头）+ data.best.account_names
评分明细	data.best.detail.details[] + missing + total_score（+ 后端若带 nim_tmp_note 会额外展示）
T0 基线卡（算法/DL 共用）	data.baseline（score/grade/indicators/scores[]）+ data.delta（score/indicators/scores）；卡片含「指标值对比」与「各指标评分（权重/T0得分/本方案得分/Δ得分）」两张表
风险提示 / 调整建议	data.best.risk_hints[] / data.best.suggestions[]
候选榜单（数据源 = data.candidates[]，按 score 降序）	
候选每项概要	candidates[i].{source(acor|de|dl), score, grade, indicators.{cet1, lcr, nsfr, roe}, x/plan, plan_table}
候选展开-摆布明细	candidates[i].plan_table[]

候选展开–月度指标	candidates[i].detail.real_engine.monthly（M1~M12 × 7 指标）
候选展开–月度调整	candidates[i].detail.real_engine.monthly_actions（同 best 结构）
候选引擎输出	candidates[i].engine.outputs
迭代轨迹（数据源 = data.trace[]，画收敛曲线）	
每代条目	trace[i].{algo:'ACOR'|'DE', iter, best}（本轮最优分）
每代 top3 方案	trace[i].top3_plans[{score, grade, indicators, plan_table, engine, detail.real_engine.monthly, detail.real_engine.monthly_actions}]
DL tab（数据源 = data.best_dl；单 dl 模式时 = data.best）	
DL 最优方案	data.best_dl：结构同 best（score/grade/indicators/plan_table/monthly/monthly_actions/matrix_12x64/final_bs_matrix/balance_matrix_full/detail/engine/risk_hints/suggestions）+ b.note（BMD 策略说明）
DL 完整账户表	data.best_dl.final_bs_matrix（37 × 67）+ data.best_dl.account_names 行表头；⚠ 前端此块表头仍按旧版 130 列（原B64+现B64）硬编码渲染，与后端 67 列不对齐，属已知遗留待修
DL 月度资产负债表	data.best_dl.balance_matrix_full（37 × n_months，DL 路线 13 月）+ data.best_dl.n_months
⚠ 瘦身后差异（本页响应为轻量版）	
detail.real_engine.cashflow	响应中**已裁掉**（400KB 大矩阵）；完整版在 data.result_file 落盘文件（/log/result–{task_id}.json）的 detail.real_engine.cashflow 下（含 final_balance_sheet：每科目 remaining_principal_360 / remaining_total_cashflows_360 / remaining_interest_360 逐月明细）
月度/引擎中间量	detail.real_engine.monthly / monthly_actions / engine / eve **保留在响应中**（本页候选/轨迹月度面板读的就是它）

评分分项明细字段（data.best.detail.details[] 每一项）

字段	含义
kpi_code	指标编码：LCR 流动性覆盖率 / NSFR 净稳定资金比例 / EVE 银行账户利率风险（越低越好）/ CET1 核心一级资本充足率 / CAR 资本充足率 / ROE 净资产收益率 / IBL 同业负债占比 / LR 存贷比（IBL、LR 当前为占位值 0，记为 missing 不参与评分）
kpi_name	指标中文名
current_value	该指标本次实测值（口径与评分输入一致：CET1/CAR/ROE/EVE 小数，LCR/NSFR/IBL/LR 倍数；EVE 已由百分数 ÷100）
score	该指标分段得分（0~500 区间内的段位分，如 300 / 450；红线段为 0）
weight	归一化权重（v14.3：LCR 20.25% / CET1 20.25% / NSFR 16.46% / EVE 15.19% / ROE 15.19% / CAR 12.66%，参评项合计 100%）
weighted_score	该指标对总分的加权贡献 = score × weight
seg_order	命中段位序号（从红线段 1 开始递增）
segment_desc	命中段位描述，如"红线<8.7%""⭐ 满分""极度过剩""预警–轻惩罚"
higher_is_better	1=越高越好（LCR/NSFR/CET1/CAR/ROE/IBL/LR）；0=越低越好（EVE）
rule_name	评分规则名

> 综合总分 = **Σ(单指标 score × weight) / Σweight**（加权平均，非简单累加；参评项权重合计为 1，故等于 weighted_score 之和）；本次未参与评分的指标见 **data.best.detail.missing**（当前为 IBL / LR）。

① 提交测算任务

项	说明
Method	POST
URL	/api/abm/calculate
Content–Type	application/json

请求体（request）字段说明

字段	类型	默认	说明	PRCP 数据字典对应（表.字段）
targets	object	**必填**	六指标目标对象本身**必须提供**（缺失直接 422）；对象内单个指标可省略，省略项回退 TargetsIn 默认（roe 0.0961 / cet1 0.09 / lcr 1.20 / nsfr 1.00 / nim 0.0155 / eve 0.057）	prcp_reverse_target.target_value（kpi_code='REG_*'）

targets.cet1	float	0.09	核心一级资本充足率目标（小数）	prcp_reverse_target.target_value（kpi_code='REG_CET1'）
targets.nim	float	0.0155	净息差目标（小数）	prcp_reverse_target.target_value（kpi_code='REG_NIM'）
targets.roe	float	0.0961	ROE 净资产收益率目标（小数，一期优化驱动指标⭐）	prcp_reverse_target.target_value（kpi_code='REG_ROE'）
targets.eve	float	0.057	EVE 经济价值变动目标（小数，越低越好）	prcp_reverse_target.target_value（kpi_code='REG_EVE'）
targets.nsfr	float	1.00	净稳定资金比例目标（小数）	prcp_reverse_target.target_value（kpi_code='REG_NSFR'）
targets.lcr	float	1.20	流动性覆盖率目标（小数）	prcp_reverse_target.target_value（kpi_code='REG_LCR'）
options.mode	string	algorithm	algorithm=双引擎同时（ACOR+DE 粗筛+SLSQP 精修，+ 深度学习路线B）；acor=单算法（仅 ACOR+DE 粗筛+SLSQP 精修）；dl=单深度学习（BMD 策略推理，直出 12×65）	prcp_reverse_scheme.algorithm（方案算法）
options.n_candidates	int	6	候选方案数量（1~20）	prcp_model_param（param_code='n_candidates', OPTIMIZER）【建议】
options.quadrature_weights	object	—	已废弃（仅保留兼容）：v14.3 起评分权重固定在 src/scoring_v143_data.py，八指标加权（LCR/CET1 各 20.3%、NSFR/EVE/ROE 15~16%、CAR 12.7%），传此参数不生效	—
options.acor	object	m=20, k=5, max_iter=15	蚁群参数（未传项取以下默认；**网关须写扁平键 acor_m/acor_max_iter，嵌套对象会被丢**）：m 蚂蚁数 20 / k 精英档案 5 / xi 0.5 / rho 0.08 / q 0.25 / **max_iter 15** / warm_start 1 / xi_decay 1 / patience 3 / parallel 1	prcp_model_param（param_code='acor_*'，OPTIMIZER）【建议】
options.de	object	np=30, max_iter=10	差分进化参数（未传项取以下默认；**网关须写扁平键 de_np/de_max_iter**）：**np 种群规模 30** / f 0.6 / cr 0.75 / **max_iter 10** / tol 1e−6	prcp_model_param（param_code='de_*'，OPTIMIZER）【建议】
options.slsqp	object	max_iter=1, ftol=1e−3, top_n=1	精修参数：max_iter（默认 **1** 次收尾迭代，0=跳过精修）/ ftol 1e−3 / top_n 1	prcp_model_param（param_code='slsqp_*'，OPTIMIZER）【建议】
options.decision_vars	object	12 dynamic + equity fixed	决策变量定义（12 个标黄末级 + 权益平衡项；**默认 equity 为 fixed，其余 12 项 dynamic**）：{key:{mode:'dynamic'|'fixed', value:倍率}}——fixed 的科目不参与算法优化（用户手动固定），dynamic 才是决策变量。示例：{"ib_money":{"mode":"fixed","value":1.05},"cb_mlf":{"mode":"fixed","value":1.0}}	prcp_sim_node_config（coa_node_id + annual_growth_rate：配置了即固定，未配置即算法优化）【建议】

⚠ **两套目标默认值，按入口区分**：**POST /api/abm/calculate**（决策中心，本表）未传 targets 时用 TargetsIn 默认 roe 0.0961 / cet1 0.09 / lcr 1.20 / nsfr 1.00 / nim 0.0155 / eve 0.057；**网关 POST**（/v1/{serviceId}/predict 与根 /）未传 targets 时用 TARGET_DEFAULTS roe 0.12 / cet1 0.10 / lcr 1.50 / nsfr 1.07 / nim 0.0155 / eve 0.08（与 GET /api/abm/config_schema 的 targets_default 一致）。做"目标达成对比"必须按实际入口取默认值，否则达标判定会错。
目标合法范围（越界 code=400 "目标参数校验不通过" + data.errors[]）：CET1 7%~15% / NIM 0.8%~5% / ROE (0,25%] / EVE (0,25%] / NSFR 85%~150% / LCR 100%~180%。

数据来源（非请求参数，推理侧内部取数）：基准余额 ← **prcp_data_balance**（coa_node_id × data_date：current_balance/avg_balance）；科目级五大引擎系数 ← **prcp_lcr/nsfr/cet1/eve/nim/roe_param**（scheme_code+node_code+data_date）；数据月份选择 ← **prcp_reverse_scheme.data_date**。这些由 model.py 内部按 DATE 环境变量取数，**不需要**决策中心传值。

响应体（response）字段说明

字段	类型	说明
code	int	0=成功；400=目标校验不通过
data.task_id	string	任务 ID（异步，用于轮询）
data.poll_interval_ms	int	建议轮询间隔 5000ms（1CPU 环境降低轮询以减轻 GIL 竞争）
result.best.indicators	object	最优方案六监管指标 + 辅助：cet1/car/lcr/nsfr/nim/eve/roe/rwa
result.best.x	array	13 维决策向量（12 资产负债 + 权益的调整倍率，与 plan_table 的 ratio 一一对应）
result.best.plan	object	摆布方案字典（科目 key → 调整后规模，单位亿元；含自动平衡项 cash_nonearning；内部使用，对外主要看 plan_table）
result.best.plan_table	array	12 个末级决策变量 + 权益：name/side/ratio(调整系数)/scale(规模亿)/mode(dynamic|fixed)
result.best.matrix_12x64	array	12 科目 × 65 期限桶的月度增量 action 矩阵（算法 12 倍率 → 增量按 24 月 × 65 桶均摊，即传给张涛引擎的输入；单位：人民币元）
result.best.final_bs_matrix	array	张涛原始输出完整账户表（37 行 × 67 列 = 科目代码 + 期末余额 + 65 桶剩余现金流；单位：人民币元）
result.best.cashflow_64_matrix	array	各科目剩余总现金流 65 桶矩阵（37 行 × 65 列；EVE 折现代吃此矩阵；单位：人民币元）
result.best.balance_matrix_full	array	完整月度资产负债表矩阵（张涛真实现金流输出：37 科目 × 25 月（含 T0），单位：人民币元；🟨 标黄决策变量科目）
result.best.account_names	array	科目名称列表（对应 balance_matrix_full 的行）
result.best.account_codes	array	科目代码列表（对应 balance_matrix_full 的行）

result.best.n_months	int	矩阵月数（算法路径 horizon=24 → 含 T0 共 25 月；DL 路径 12 月 → 13 月）
result.best.score / grade	float/string	综合得分（0–500 分制，v14.3 八指标加权）/ 评级：≥400 **A**、≥300 **B**、≥200 **C**、其余 **D**（算法与 DL 同一口径）
result.best.detail	object	评分分项（**响应为轻量版**）：details[]（每指标 current_value/score/seg_order/segment_desc/higher_is_better，字段见上方明细表）/missing（未参与评分指标）/total_score(500分制)/total_score_100(旧100分制)；红线否决时 red_line=true。⚠️ **不含 real_engine.cashflow**（原 400KB 现金流行矩阵已从响应裁掉，全量见 data.result_file 落盘文件）；real_engine 的 engine / monthly / monthly_actions / eve 仍在
result.best.engine.inputs	object	给五大引擎的输入：groups(12 末级决策变量明细)/fixed(固定科目)/balance(平衡科目)/summary
result.best.engine.outputs	object	五大引擎输出：cet1/nim/lcr/nsfr/roe/eve/structure 中间量 + nii（NII 净利息收入，亿元）
result.best.monthly	object	月度指标（纯展示，不参与评分择优）：{cet1/nim/lcr/nsfr/roe/nii/eve 各 12 个值（M1~M12)}；ROE=月环比x12 年化，EVE=月末估值日横截面，NIM/NII=逐月利息；候选/轨迹同理见 candidates[i].detail.real_engine.monthly、trace[].top3_plans[].detail.real_engine.monthly（响应中保留）
result.best.monthly_actions	array[12]	月度科目调整明细（纯展示）：每月 [{month, items:[{account_code, account_name, ratio(算法=倍率/DL=月增率), t0_balance_yi, monthly_increment_yi, cum_increment_yi}]}]；候选/轨迹同见 candidates[i].detail.real_engine.monthly_actions、trace[].top3_plans[].detail.real_engine.monthly_actions（响应中保留）
result.best_dl	object|null	DL 路线最优方案（algorithm 模式与算法 best 分开返回；结构同 best，含 indicators/score/detail/engine/monthly/monthly_actions/matrix_12x64 等；detail 同为轻量版）；单 dl 模式时 best 即 DL 方案、best_dl 与 best 同值；纯 acor 模式为 null
result.best.risk_hints	array	风险提示文案（按指标实际值触发）
result.best.suggestions	array	改进建议文案（按指标实际值与目标差距触发）
result.candidates	array	候选榜单（含 source=精修起点来源 acor|de、x/plan（13 维决策向量，plan 为兼容别名）、plan_table/indicators/score/grade/detail（轻量版：仅 real_engine.cashflow 裁掉走 result_file，monthly/monthly_actions/engine 保留）/engine.outputs）
result.mode / result.timestamp	string	本次测算模式（algorithm|acor|dl）/ 结果生成时间
result.reqid	string|null	网关提交时带的 reqid 原样回传（对账用；与 baseline/delta 一样不受 fields 裁剪）
result.baseline	object	T0 基线（**算法 / DL 两条路线共用同一份**）：T0 原样（12 个决策倍率全 1.0 = 不做任何投放）走完整评分链路的结果 {label, ratios, score(总分), grade, **scores[](单指标评分)**, score_formula, indicators, detail(轻量), engine, monthly, monthly_actions}；进程内只算一次（首次约 2~5s，之后命中缓存 0s）；**不受 fields 白名单裁剪**
result.baseline.scores	array	T0 基线**各指标评分明细**（与 result.best.detail.details 同构，每项 {kpi_code, kpi_name, value, score, weight, weighted_score, seg_order, segment_desc, higher_is_better, rule_name}）；value 口径与评分输入一致（CET1/CAR/ROE/EVE 为小数，LCR/NSFR/IBL/LR 为倍数）；weight 为归一化权重（合计 1.0）；未参与的指标见 missing。**总分 score = Σ(score×weight) / Σweight（求和恒等于 score）**
result.baseline.score_formula	string	总分计算口径说明文案（前端直接展示）：本轮为 v14.3 八指标加权，满分 500
result.delta	object	本方案相对 T0 基线的差值：{score: 分差（正=优于基线），indicators: {cet1/lcr/nsfr/roe/eve/nim 各自指标值差值}, scores: {kpi_code: 单指标得分差（正=该指标得分高于 T0)}}；与 baseline 一样**不受 fields 白名单裁剪**
result.trace	array	粗筛迭代轨迹（**轻量版**）：每代 {algo:'ACOR'|'DE', iter, best, top3_plans:[{score/grade/plan_table/indicators/detail(轻量)/engine}]}——画收敛曲线取 algo/iter/best；每代完整评分+五大引擎明细在 result_file 的全量 trace 里
result.result_file	string	本次任务**全量 payload** 落盘路径（容器/NAS 内，LOG_DIR/result-{task_id}.json，如 /log/result-90c7a91c.json）。响应瘦身（网关 ~4MB 缓冲限制）：trace/candidates/best/best_dl 的 detail 里 **real_engine.cashflow（现金流行矩阵，400KB 大头）不随响应回传**，只在落盘文件里；real_engine 的 engine/monthly/monthly_actions/eve 与评分明细（details 等）响应中仍完整保留

② 查询进度 / ③ 取结果 / ④ 取消任务

操作	接口	说明
进度（网关 POST）	POST /v1/{serviceId}/predict，body {"action":"progress","task_id":"..."}	返回 {stage, percent, message, done, success, cancelled, created_at, updated_at, **history_count, history[最近50条]**}，前台每 2~5s 轮询；done:true,success:true 再取结果
进度（本地 GET）	GET /api/abm/progress/{task_id}	同网关 POST，另带 history_count + history（最近 50 条）；任务不存在时是 HTTP 404 + {"detail":"任务不存在"}（与网关的 code=404 结构不同）
进度历史全量	GET /api/abm/progress/{task_id}/history	返回 {history_count, history:全部}——每条 {stage, percent, message, ts:时间}，即进度历史记录完整清单（网关未配 GET 时需直连容器或走落盘）
结果	POST /v1/{serviceId}/predict，body {"action":"result","task_id":"...", **可选 "fields":"best|candidates"**（竖线分隔)}（本地 GET /api/abm/result/{task_id}，query **?fields=best|candidates**）	完成返回 code=0 + data=result（含 result_file 落盘路径）；未完成 code=202 带 percent；失败 code=500 看 message；任务不存在 code=404；**fields 白名单**：只返回指定顶层键（如 "best" / "best|candidates" / "trace"，恒带 reqid/baseline/delta），每块远小于网关 4MB 缓冲；不传=全量（默认双引擎全量 ~4.3MB 会超限，建议按需取）
取消	POST /api/abm/cancel/{task_id}	请求取消正在运行的任务（算法每轮迭代前检查取消标志，安全退出）
健康	GET /api/abm/health	服务状态
配置	GET /api/abm/config_schema	可设置项清单 + 默认值 + targets_default（驱动表单，也用于对端取目标默认值做达成对比）

curl 调用示例（本地直连）

```
curl -X POST http://<host>:8000/api/abm/calculate \
  -H "Content-Type: application/json" \
  -d '{
    "targets": {"cet1": 0.09, "nim": 0.0155, "roe": 0.10, "eve": 0.057, "nsfr": 1.00, "lcr": 1.20},
    "options": {"mode": "algorithm", "n_candidates": 6}
  }'
# 返回 → 取 data.task_id 轮询:
curl http://<host>:8000/api/abm/progress/{task_id}
curl http://<host>:8000/api/abm/result/{task_id}
```

curl 调用示例（Windows cmd / 网关，三步全用同一个 POST）

```
REM ① 提交（返回 data.task_id）
curl -XPOST http://28.4.159.168:10000/v1/cabs-alm-optimization/predict -H "Content-Type: application/json" -d "{\"targets\":{\"roe\":0.0961,\"cet1\":0.09,\"lcr\":1.2,\"nsfr\":1,\"nim\":0.0155,\"eve

REM ② 查进度（done:true,success:true 后再取结果；history 为最近 50 条进度历史）
curl -XPOST http://28.4.159.168:10000/v1/cabs-alm-optimization/predict -H "Content-Type: application/json" -d "{\"action\":\"progress\",\"task_id\":\"\"}"

REM ③ 取结果（code:0 + data；含 data.result_file=/log/result-.json 全量落盘路径）
curl -XPOST http://28.4.159.168:10000/v1/cabs-alm-optimization/predict -H "Content-Type: application/json" -d "{\"action\":\"result\",\"task_id\":\"\"}"

REM ④ 取结果（fields 白名单按需拆，避免全量超网关 4MB；返回 data = 指定键 + reqid）
REM    fields:["best"] ≈0.55MB / ["candidates"] ≈1.35MB / ["trace"] ≈1.85MB / ["best_dl"] ≈0.55MB（默认双引擎口径）
curl -XPOST http://28.4.159.168:10000/v1/cabs-alm-optimization/predict -H "Content-Type: application/json" -d "{\"action\":\"result\",\"task_id\":\"\",\"fields\":[\"best\"]}"
REM    多键逗号串也行: \"fields\":\"best,candidates\"
curl -XPOST http://28.4.159.168:10000/v1/cabs-alm-optimization/predict -H "Content-Type: application/json" -d "{\"action\":\"result\",\"task_id\":\"\",\"fields\":\"best,candidates\"}"
```